Assignment: Build "LeaveDesk", an HR Leave MCP Server, from Scratch

Assignment Brief | Domain: HR operations (Kaveri Infotech, a fictional company) | Reference code: github.com/SetuAI/Oracle_26_30_Oct (folders MCP_foundational/ and amfi-mcp-server/)

Level	Time	Stack	You hand in
Beginner to Intermediate	4 - 6 hours	Python 3.10+, MCP Python SDK 1.x (FastMCP), MCP Inspector (Node.js), VS Code + GitHub Copilot Agent mode or Claude Desktop	3 Python files + mcp.json + notes.md + screenshots

The goal in one line. You will build an MCP server that gives an AI assistant four tools, one resource and one prompt over a small HR system, test it by hand in the MCP Inspector, call it from your own Python client, and finally connect it to a real AI app that answers employees' leave questions correctly. In class you saw a server with one tool (the claims server). This assignment adds a calculation tool, a rule-checking tool, the two other MCP building blocks (resources and prompts) and a client you write yourself.

1. The scenario

Priya runs the HR operations desk at Kaveri Infotech, a 400-person IT services company in Pune. Every day employees message her team the same questions: "*How many casual leaves do I have left?*", "*Is my leave request approved?*", "*If I take 9 to 13 November off, how many leave days does it cost me?*", "*Can I take three days of casual leave next week?*". Priya wants an AI assistant inside VS Code and Claude Desktop that answers these instantly and correctly.

There are two real difficulties. First, the AI model has never seen Kaveri Infotech's HR records: it cannot know any employee's balance, any request's status or the company's holiday list. Second, even with the holiday list, a model counting "working days between two dates, skipping weekends and holidays" in its head makes mistakes. So the counting must be done by code, and the AI must call that code. Your job is to build the MCP server that offers that code to the AI.

2. What happens when an employee asks a question

#	Who acts	What happens	Comes out
0	AI app (host)	When VS Code or Claude Desktop starts, it reads mcp.json, launches python leave_server.py, and runs the MCP handshake (initialize).	An open connection to your server
1	MCP client inside the app	Sends tools/list, resources/list and prompts/list.	4 tool descriptions, 1 resource, 1 prompt
2	AI model	Reads the employee's question plus the 4 tool descriptions and decides which tool to call and with which arguments.	e.g. check_leave_eligibility(EMP-102, casual, 2026-11-11, 2026-11-13)
3	AI app	Shows the user an Allow / Continue button for the tool call.	User approval
4	MCP client	Sends tools/call with the arguments to your server.	A request on stdin of leave_server.py
5	Your server	Runs the matching function in leave_data.py and returns its dict as JSON.	{"eligible": false, "reasons": [...]}
6	AI model	Writes the reply in plain English using only that JSON.	"No - you need 3 days but have 1 left."

If the employee ID does not exist, step 5 returns **found: false**, and the model must say so instead of inventing a balance. If the server crashes or prints junk to stdout, step 4 fails and the app shows no tools. That is why you test each layer on its own before connecting the next (Build Guide, Steps 1 to 4).

3. What you will build

MCP building block	Name	Inputs	Returns / does
Tool 1 (lookup)	get_leave_balance	employee_id	Name, department, days left per leave type; found: false if unknown
Tool 2 (filtered lookup)	list_leave_requests	employee_id, status (optional)	Matching requests with ID, type, dates, status
Tool 3 (calculation)	count_working_days	start_date, end_date (YYYY-MM-DD)	Calendar days, working days, holidays skipped
Tool 4 (rules check)	check_leave_eligibility	employee_id, leave_type, start_date, end_date	eligible true/false + one reason per broken rule

MCP building block	Name	Inputs	Returns / does
Resource	policy://leave	none (read by URI)	The leave policy text, loaded as context
Prompt	plan_my_leave	employee_id, start_date, end_date	A ready-made instruction that tells the AI to use tools 1, 3 and 4
Client	client.py	none	Your own program that lists and calls all of the above, with no AI

Tool, resource or prompt - who decides? A **tool** is called when the *AI model* decides it needs it. A **resource** is read-only content that the *app or user* attaches as context (in VS Code: Add Context, then MCP Resources). A **prompt** is a template the *user* picks (in VS Code: type / in chat). You will see all three in the Inspector's three tabs.

4. The data you must use

All data is fictional and is given in full in the Build Guide (Step 1). Type it exactly as given: the expected answers in the acceptance tests depend on it. Summary: 4 employees (EMP-101 to EMP-104) with casual, sick and earned balances; 5 leave requests (LR-5001 to LR-5005) with statuses Pending, Approved, Rejected; 4 company holidays (9 Nov, 10 Nov, 24 Nov and 25 Dec 2026); a 5-rule leave policy, of which the code checks three: enough balance, casual leave at most 3 consecutive working days, earned leave needs 7 days' notice.

5. Rules for this assignment

- **Pin the SDK below version 2:** `mcp[cli]>=1.2,<2`. MCP Python SDK 2.x (released 2026) renamed `FastMCP` to `MCPServer`; the class code and this assignment use 1.x.
- **Two files, two jobs.** `leave_data.py` holds data and plain functions and must not import mcp. `leave_server.py` only wraps those functions; it holds no data.
- **All tools are read-only.** No tool may change data. (Writing tools are a stretch goal, with a confirmation step.)
- **Every tool docstring says when to use it** ("Use this whenever..."), describes every argument with an example, and says what is returned when nothing is found. The model reads these words to choose tools.
- **Never print() in leave_server.py.** In stdio transport, stdout carries the MCP messages; a stray print breaks the connection.
- **Honest results.** Unknown IDs return found: false; bad dates raise a clear error message; nothing is invented.

6. What to submit and how it is graded

Item	Marks	What earns full marks
leave_data.py	15	Checkpoint 1 output matches exactly; no mcp import; clear error on a bad date
leave_server.py	25	Checkpoint 2 passes: 4 tools, 1 resource, 1 prompt visible in the Inspector; complete docstrings; no print()
client.py	15	Checkpoint 3 passes: lists tools, resources, prompts; calls 2 tools; reads the resource; gets the prompt
AI host connection	20	Acceptance tests A1 - A9 run in VS Code or Claude Desktop; correct tool chosen; screenshots for A3, A4, A7 and A9
notes.md	15	Experiments E1 - E3 recorded with what you changed, what the model did, and why
Code quality	10	Header comment on each file; constants not repeated; runs from a clean clone; no secrets or .venv committed

Submit: a zip or GitHub link with leave_data.py, leave_server.py, client.py, requirements.txt, .vscode/mcp.json (or your Claude Desktop config snippet), notes.md and a screenshots/ folder (Inspector tools list, one Inspector tool result, client.py output, and the four acceptance-test screenshots).

7. Stretch goals (optional, no extra marks)

- Add a write tool `apply_leave(...)` that first calls check_leave_eligibility and only adds a Pending request when eligible. Ask the AI to use it and note when the app asks for confirmation. Why do write tools need more care than read tools?
- Write `ai_client.py`: your client plus an OpenAI model. Convert each tool's inputSchema into an OpenAI function definition, let the model choose, call the tool with session.call_tool, send the result back, and print the answer.
- Run the server over HTTP with `mcp.run(transport="streamable-http")` and connect the Inspector with Transport = Streamable HTTP. What changes, and what stays the same?